

Cloud Computing & DevOps

Easy Exam Notes - C-DAC Module

Git | Cloud Basics | Architecture | Virtualization | SaaS-PaaS-IaaS | AWS EC2 & Lambda | Dashboard |
Deployment | AI & Analytics | DevOps

How to use these notes

1. Read each topic once with the simple explanation.
2. Learn the tables and 'Exam tip' boxes - these are most asked.
3. Practise the commands and the EC2 / Lambda steps.
4. Finish with the Cheat Sheet and MCQs at the end.

#	Topic
1	Git and Version Control
2	Cloud Computing Basics
3	Cloud Architecture
4	Virtualization
5	SaaS vs PaaS vs IaaS
6	Cloud Vendors: AWS Products
7	EC2 (Elastic Compute Cloud)
8	AWS Lambda
9	AWS Dashboard / Console
10	Deploying an Application on Cloud
11	AI and Analytics Workloads
12	Introduction to DevOps
13	Cheat Sheet and Practice MCQs

1. Git and Version Control

What is Version Control?

A **Version Control System (VCS)** keeps a history of every change made to your files. You can go back to an old version, see who changed what, and many people can work on the same project without overwriting each other.

EASY WAY TO REMEMBER: Think of 'Save As' with a full history - like Google Docs version history, but for code.

Types of VCS

Type	Meaning	Example	Problem / Advantage
Local	Versions saved on one computer	RCS	Not shareable
Centralized (CVCS)	One central server; users check out files	SVN, CVS	Server down = nobody can work (single point of failure)
Distributed (DVCS)	Every user has a full copy of the repo	Git, Mercurial	Works offline, fast, no single point of failure

Git vs GitHub

- **Git** = the tool (software) installed on your computer. Created by Linus Torvalds in 2005.
- **GitHub / GitLab / Bitbucket** = websites that host Git repositories online (the remote).

The Git areas (very important)

Working Directory (your files) --git add--> Staging Area (ready list) --git commit--> Local Repo (.git folder) --git push--> Remote (GitHub)

Basic Git commands

```
git config --global user.name "Your Name"      # set name
git config --global user.email "you@mail.com"   # set email
git init                                         # create a new repo in current folder
git clone <url>                                 # download a remote repo
git status                                      # what changed?
git add file.txt                               # stage one file (git add . = stage all)
git commit -m "message"                       # save a snapshot with a message
git log --oneline                              # show history in short form
git diff                                        # show unstaged changes
```

Branching and merging

```
git branch                                     # list branches
git branch feature1                           # create branch
git checkout feature1                         # switch to it (or: git switch feature1)
git checkout -b feature1                     # create + switch in one step
git merge feature1                           # merge feature1 into the CURRENT branch
git branch -d feature1                       # delete branch
```

Working with remote

```
git remote add origin <url>                 # connect local repo to GitHub
git push -u origin main                      # upload commits
git pull                                     # download + merge (fetch + merge)
git fetch                                    # download only, no merge
```

Undo and extra commands

```
git stash / git stash pop                    # temporarily hide / bring back changes
git revert <commit-id>                      # SAFE undo: makes a new opposite commit
```

```
git reset --hard <commit-id> # DANGEROUS: deletes changes permanently
git tag v1.0                  # label a release
```

Important terms

Term	Simple meaning
Branch	A separate line of work so main code stays safe
Merge conflict	Same line edited in two branches. Open file, fix manually, then git add + git commit
Fork	Your own copy of someone else's GitHub repo
Pull Request (PR)	Request to merge your changes into the original repo
.gitignore	File listing things Git should NOT track (e.g., node_modules/, .env)
HEAD	Pointer to your current commit / branch
Rebase vs Merge	Merge keeps history + merge commit. Rebase makes history a straight line
fetch vs pull	fetch = only download. pull = fetch + merge
revert vs reset	revert = safe (new commit). reset = rewrites history (dangerous)

EXAM TIP: Typical workflow: edit -> git add -> git commit -> git push -> Pull Request -> review -> merge.

2. Cloud Computing Basics

Definition

Cloud computing means using computing resources - servers, storage, databases, networking, software - over the **internet** on a **pay-as-you-use** basis, instead of buying and maintaining your own hardware.

NIST definition: A model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort.

EASY WAY TO REMEMBER: Electricity: you do not build a power plant at home. You plug in and pay for the units you use. Cloud = electricity for computing.

5 Characteristics (NIST) - always asked

Characteristic	Meaning	Example
On-demand self-service	Get resources yourself, any time, no calling the provider	Launch an EC2 server in 2 minutes
Broad network access	Access from anywhere using any device	Open Gmail from phone or laptop
Resource pooling	Provider's resources shared by many customers (multi-tenancy)	Many companies on same physical server
Rapid elasticity	Scale up/down quickly, often automatically	Server count rises on sale day, drops after
Measured service	Usage is metered; you pay for what you use	Pay per hour of EC2

EXAM TIP: Memory trick: S-B-P-R-M = Self-service, Broad access, Pooling, Rapid elasticity, Measured service.

Components of Cloud

- **Clients:** devices/browsers used to access the cloud.
- **Data centres and servers:** the physical machines.
- **Virtualization layer:** hypervisor that creates virtual machines.
- **Storage:** object, block and file storage.
- **Network:** internet, load balancers, VPN.
- **Management software:** provisioning, monitoring, billing, orchestration.
- **Security layer:** identity, encryption, firewalls.

Cloud Provider

A company that owns data centres and rents out cloud services. Examples: **AWS, Microsoft Azure, Google Cloud (GCP), IBM Cloud, Oracle Cloud, Alibaba Cloud**. In India: Yotta, Tata Communications, Sify, CtrlS, NIC MeghRaj (government cloud).

How to choose a provider: cost, security and compliance, reliability (SLA), services offered, data-centre location, support, and vendor lock-in risk.

Deployment models

Model	Meaning	Example
Public	Owned by provider, shared by many customers over internet	AWS, Azure, GCP
Private	Used by only one organization (on-premise or hosted)	OpenStack, VMware, bank's own cloud

Model	Meaning	Example
Hybrid	Public + private connected together	Sensitive data on private, website on public
Community	Shared by organizations with common needs	Government departments, hospitals
Multi-cloud	Using more than one public provider	AWS + Azure together

Cloud bursting: in hybrid cloud, when private cloud is full, extra load 'bursts' to the public cloud.

Organizational scenarios (where companies use cloud)

Organization	How cloud helps
Startup	No big hardware cost; start small, grow fast
E-commerce	Auto-scale during festival sales, scale down later
Bank / Healthcare	Private or hybrid cloud for security and compliance
Media / OTT	Store huge videos, deliver quickly through CDN
Education	Virtual labs, online classes, LMS
Government	Community/private cloud (MeghRaj)
Software company	Create and delete test environments on demand; CI/CD
Any company	Backup and disaster recovery in another region
AI / Data teams	Rent GPUs and big clusters only when needed

Benefits and Limitations

Benefits	Limitations
No upfront cost (CapEx becomes OpEx)	Needs good internet connection
Scalable and elastic	Security and privacy concerns
Pay only for what you use	Vendor lock-in (hard to switch)
Fast deployment	Less control and customization
Global reach, high availability	Downtime is outside your control
Automatic updates and backups	Data-residency and compliance issues
Easy disaster recovery and collaboration	Cost can grow if not monitored

3. Cloud Architecture

Cloud architecture is how all cloud components are arranged and connected. It has two main parts joined by a network.

FRONT END	<---- Internet / Network ---->	BACK END
(Client side)		(Cloud side)
Browser, mobile app		Servers, storage, VMs,
user interface		security, management

- **Front end:** what the user sees and uses (browser, app).
- **Back end:** servers, storage, virtual machines, databases, security and the management software.
- **Network:** internet / intranet that connects front end and back end.
- **Management (orchestration) layer:** handles provisioning, monitoring, billing, automation.

Cloud service stack

SaaS - Applications (Gmail, Zoom)
PaaS - Runtime, middleware, database
IaaS - Virtual machines, storage, network
Virtualization (Hypervisor)
Physical hardware (Data centres)

Key architecture concepts

Concept	Simple meaning
Multi-tenancy	Many customers share the same hardware but their data is isolated
Scalability	Ability to handle more load. Vertical = bigger machine (scale up). Horizontal = more machines (scale out)
Elasticity	Automatic scale up/down as demand changes
Load balancing	Spread incoming traffic across many servers
High availability	No single point of failure; use multiple Availability Zones
Fault tolerance	System keeps working even if a part fails
SLA	Service Level Agreement: promised uptime (e.g., 99.99%) and penalties
Loose coupling	Parts work independently, connected by queues (SQS)

Typical 3-tier architecture on cloud

Users -> DNS (Route 53) -> Load Balancer -> Web/App servers (Auto Scaling)
|
Database (RDS Multi-AZ) + S3 + CloudFront CDN

EXAM TIP: Scalability = ability to grow. Elasticity = growing AND shrinking automatically. Vertical = scale up, Horizontal = scale out.

4. Virtualization

Virtualization means creating a virtual version of a computer (hardware, OS, storage or network) so that **many virtual machines (VMs) run on one physical machine.**

EASY WAY TO REMEMBER: One big building (physical server) divided into many separate flats (VMs). Each flat is private, but they share the same building.

Hypervisor (VMM)

Software that creates and runs virtual machines.

	Type 1 (Bare-metal)	Type 2 (Hosted)
Runs on	Directly on hardware	On top of a host OS
Speed	Faster, better performance	Slower (extra OS layer)
Used for	Data centres, cloud	Desktop, learning, testing
Examples	VMware ESXi, Hyper-V, Xen, KVM	VirtualBox, VMware Workstation

Types of virtualization

- **Server:** many VMs on one server.
- **Storage:** many disks appear as one pool (SAN/NAS).
- **Network:** virtual networks (VLAN, SDN, VPN).
- **Desktop:** remote virtual desktops (VDI).
- **Application / OS-level:** containers such as Docker.

Techniques

- **Full virtualization:** guest OS is unchanged; hypervisor handles everything.
- **Paravirtualization:** guest OS is modified to talk to hypervisor (Xen).
- **Hardware-assisted:** CPU support (Intel VT-x, AMD-V).

VM vs Container

Virtual Machine	Container
Has its own full guest OS	Shares host OS kernel
Size in GBs, boots in minutes	Size in MBs, starts in seconds
Strong isolation	Lighter isolation
Needs hypervisor	Needs container engine (Docker)

Benefits: better hardware use, isolation, snapshots, live migration, quick provisioning, lower cost, easy backup. **Virtualization is the base technology of cloud** - it enables resource pooling, multi-tenancy and elasticity.

5. SaaS vs PaaS vs IaaS

Feature	IaaS	PaaS	SaaS
Full form	Infrastructure as a Service	Platform as a Service	Software as a Service
You get	Virtual machines, storage, network	Platform to build and run apps	Ready-made software
Used by	System admins, IT teams	Developers	End users
You manage	OS, runtime, apps, data	Only your app and data	Nothing (just use it)
Provider manages	Hardware, virtualization	Plus OS, runtime, middleware	Everything
Control	Highest	Medium	Lowest
Examples	AWS EC2, Azure VM, Google Compute Engine	Elastic Beanstalk, Google App Engine, Heroku	Gmail, Salesforce, Office 365, Zoom

EASY WAY TO REMEMBER: Pizza analogy. IaaS = buy ingredients and make pizza at home. PaaS = order a base, add your own toppings. SaaS = eat at a restaurant, everything is done for you.

Who manages what

Layer	On-premise	IaaS	PaaS	SaaS
Application	You	You	You	Provider
Data	You	You	You	Provider
Runtime / Middleware	You	You	Provider	Provider
Operating System	You	You	Provider	Provider
Virtualization, Servers, Storage, Network	You	Provider	Provider	Provider

Other models: FaaS (Function as a Service - Lambda), CaaS (Containers as a Service - ECS/EKS), DBaaS (Database - RDS), XaaS (Anything as a Service).

6. Cloud Vendors: AWS Products and Solutions

AWS (Amazon Web Services) is the world's largest cloud provider, launched in 2006, offering 200+ services.

AWS global infrastructure

- **Region:** a geographic area, e.g., Mumbai (ap-south-1), Hyderabad (ap-south-2). Contains multiple AZs.
- **Availability Zone (AZ):** one or more separate data centres inside a region. Use 2+ AZs for high availability.
- **Edge Location:** small sites used by CloudFront CDN to deliver content faster to users.

Main AWS services by category

Category	Services
Compute	EC2, Lambda, Elastic Beanstalk, ECS, EKS, Fargate, Lightsail
Storage	S3 (object), EBS (block), EFS (file), Glacier (archive)
Database	RDS, Aurora, DynamoDB (NoSQL), ElastiCache, Redshift
Networking	VPC, Route 53 (DNS), CloudFront (CDN), ELB, Direct Connect
Security	IAM, KMS, WAF, Shield, GuardDuty

Category	Services
Management	CloudWatch, CloudTrail, CloudFormation, Config
Analytics	Athena, EMR, Glue, Kinesis, QuickSight
AI / ML	SageMaker, Rekognition, Comprehend, Polly, Lex, Bedrock
DevOps	CodeCommit, CodeBuild, CodeDeploy, CodePipeline
Integration	SNS (notifications), SQS (queue), API Gateway, EventBridge

Compute products and services

- **EC2:** virtual servers (IaaS).
- **Lambda:** run code without servers (serverless / FaaS).
- **Elastic Beanstalk:** upload code, AWS handles deployment (PaaS).
- **ECS / EKS / Fargate:** run Docker containers / Kubernetes.
- **Lightsail:** simple, low-cost virtual server for small apps.
- **Auto Scaling and ELB:** add/remove servers automatically and balance traffic.

Storage in one table

	S3	EBS	EFS
Type	Object storage	Block storage (like a hard disk)	File storage (shared folder)
Attached to	Accessed via internet/API	One EC2 in same AZ	Many EC2 at once
Use	Images, backups, websites	OS disk, databases	Shared files for many servers

Other vendors (for comparison)

Service	AWS	Azure	Google Cloud
Virtual machine	EC2	Virtual Machines	Compute Engine
Serverless	Lambda	Functions	Cloud Functions
Object storage	S3	Blob Storage	Cloud Storage
PaaS	Elastic Beanstalk	App Service	App Engine
Kubernetes	EKS	AKS	GKE
ML platform	SageMaker	Azure ML	Vertex AI

7. EC2 - Elastic Compute Cloud

EC2 gives you **virtual servers (called instances)** in the AWS cloud. You choose the OS, CPU, RAM and storage, and pay only for the time you use. It is an **IaaS** service. 'Elastic' means you can increase or decrease the number of servers quickly.

Key terms

Term	Meaning
Instance	A virtual server
AMI (Amazon Machine Image)	Template containing OS + software used to launch an instance
Instance type	Size/power of the server (CPU, RAM, network). Example: t2.micro, t3.micro (free tier)
Key pair	Public/private key (.pem file) used to log in via SSH
Security Group	Virtual firewall for the instance. Stateful, allow rules only
EBS volume	Permanent disk attached to instance
Instance store	Temporary disk; data lost when instance stops/terminates
Elastic IP	Fixed public IP address that does not change
User data	Script that runs automatically at first boot
Snapshot	Backup of an EBS volume
Tags	Name labels (key=value) to organize resources and track cost

Instance type families

Family	Optimized for	Examples
General purpose	Balanced CPU/RAM (web servers)	t3, m5
Compute optimized	High CPU (batch, gaming)	c5
Memory optimized	High RAM (in-memory DB)	r5
Storage optimized	High disk speed (big data)	i3
Accelerated (GPU)	Machine learning, graphics	p3, g4

Pricing models

Model	Meaning	Best for
On-Demand	Pay per second/hour, no commitment	Short or unpredictable work
Reserved / Savings Plan	Commit for 1 or 3 years, big discount	Steady, long-running apps
Spot	Unused capacity, up to ~90% cheaper, AWS can take it back	Flexible jobs (batch, training)
Dedicated Host	A whole physical server just for you	Licensing / compliance

Instance states

pending -> running -> stopping -> stopped -> (start again) running
 \ -> shutting-down -> terminated (deleted permanently)

- **Stopped:** no compute charge, but EBS storage is still charged. Public IP changes unless Elastic IP is used.
- **Terminated:** instance is deleted and cannot be recovered.

Steps to launch an EC2 instance (write this in exam)

- 1. Log in to AWS Console, open **EC2**, click **Launch instance**.
- 2. Enter a **name** and tags.
- 3. Choose an **AMI** (Amazon Linux or Ubuntu).
- 4. Choose **instance type** (t2.micro for free tier).
- 5. Create or select a **key pair** and download the .pem file.
- 6. **Network settings**: choose VPC/subnet and a **Security Group** (allow SSH port 22 and HTTP port 80).
- 7. Configure **storage** (EBS, e.g., 8 GiB).
- 8. (Optional) paste a **user data** script.
- 9. Click **Launch instance** and wait until state is 'running'.
- 10. Connect using SSH.

```
chmod 400 mykey.pem
ssh -i mykey.pem ec2-user@<public-ip>      # Ubuntu AMI: ubuntu@<public-ip>
```

Host a simple website on EC2

```
sudo yum update -y
sudo yum install -y httpd                # Apache web server (Ubuntu: apt install apache2)
sudo systemctl start httpd
sudo systemctl enable httpd              # start automatically on reboot
echo "<h1>Hello from EC2</h1>" | sudo tee /var/www/html/index.html
```

Open `http://<public-ip>` in a browser. If it does not open, check that **port 80 is allowed in the Security Group**.

Same thing using User Data (runs at boot):

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>Hello from EC2</h1>" > /var/www/html/index.html
```

Scaling EC2

Put EC2 instances in an **Auto Scaling Group (ASG)** behind an **Application Load Balancer (ALB)**. ASG adds or removes instances automatically based on rules (e.g., CPU above 70%).

EXAM TIP: Questions often ask: AMI, key pair, security group, pricing models, stopped vs terminated, and launch steps.

8. AWS Lambda (Serverless)

Lambda lets you run your code **without managing any servers**. You upload a function; AWS runs it only when an event happens and you pay only for the number of requests and the time it runs (billed per millisecond).

EASY WAY TO REMEMBER: EC2 = rent a car and pay while you own it. Lambda = take a taxi, pay only for the trip.

Key points

- **Event-driven:** runs when triggered by S3 upload, API Gateway request, schedule (EventBridge), SNS/SQS message, DynamoDB change.
- **Languages:** Python, Node.js, Java, Go, .NET, Ruby.
- **Handler:** the entry-point function that Lambda calls.
- **Execution role (IAM):** permissions the function has (e.g., to read S3).
- **Stateless:** nothing is remembered between runs; store data in S3 or DynamoDB.
- **Limits:** maximum 15 minutes per run; memory 128 MB to 10 GB.
- **Cold start:** small delay when a new environment starts for the first time.
- **Auto scaling:** runs many copies in parallel automatically.
- **Free tier:** 1 million requests per month.

Sample Python function

```
import json

def lambda_handler(event, context):
    name = event.get("name", "World")
    return {
        "statusCode": 200,
        "body": json.dumps({"message": "Hello, " + name + "!"})
    }
```

Create a Lambda function (steps)

1. Console -> **Lambda** -> **Create function**.
2. Choose **Author from scratch**; give a name, runtime (Python) and role.
3. Write the code in the editor and click **Deploy**.
4. Click **Test**, create a test event, and view the result/logs.
5. Add a **trigger** (API Gateway, S3, schedule) to run it automatically.

Use cases

- Resize images when uploaded to S3.
- Backend for REST APIs (with API Gateway).
- Scheduled jobs (daily cleanup, reports).
- Process files or streaming data in real time.
- Chatbots and automation.

EC2 vs Lambda

EC2	Lambda
You manage the server (OS, patches)	No server management
Runs all the time	Runs only when triggered

EC2	Lambda
Pay per hour/second while running	Pay per request + run time
Good for long-running apps	Good for short tasks (max 15 min)
Manual or Auto Scaling	Automatic scaling

9. Dashboard - AWS Management Console

The **AWS Management Console** is the web interface to use all AWS services. Other ways to access AWS: **CLI** (command line) and **SDK/API** (from code).

What you see on the dashboards

- **Console Home:** recently visited services, favourites, cost and usage widgets, health notifications.
- **Region selector (top-right):** choose the region (e.g., Mumbai). Resources are region-specific - if you cannot see your instance, check the region first.
- **EC2 Dashboard:** number of running instances, volumes, key pairs, security groups, Elastic IPs, snapshots, load balancers. Has the **Launch instance** button. The left menu has Instances, Images (AMIs), Elastic Block Store, Network and Security, Load Balancing, Auto Scaling.
- **CloudWatch dashboard:** graphs of CPU, network and disk usage; set alarms (e.g., email when CPU is above 80%).
- **Billing and Cost Management:** see your spending, Cost Explorer, Budgets and alerts.
- **Account menu:** security credentials, billing, MFA, switch role.

EXAM TIP: First step after creating an AWS account: enable MFA on the root user and create an IAM user. Do not use the root account for daily work.

10. Deploy an Application over the Cloud

General steps

- 1. **Choose the service model:** EC2 (IaaS), Elastic Beanstalk (PaaS), Lambda (serverless) or containers (ECS/EKS).
- 2. **Set up network:** VPC, subnets, security groups.
- 3. **Create compute and storage:** EC2 + EBS / S3.
- 4. **Install runtime and web server**, then bring your code (use `git clone`).
- 5. **Add a database:** RDS or DynamoDB.
- 6. **Add load balancer and Auto Scaling** for traffic and high availability.
- 7. **DNS and HTTPS:** Route 53 and an SSL certificate (ACM).
- 8. **Monitor:** CloudWatch metrics, logs, alarms.
- 9. **Automate:** CI/CD pipeline and Infrastructure as Code.

Example: deploy a Python / Node app on EC2

```
sudo yum install -y git python3 python3-pip
git clone https://github.com/you/myapp.git
cd myapp
pip3 install -r requirements.txt
python3 app.py # or run with gunicorn / pm2 so it keeps running
```

Open the app's port (e.g., 5000 or 3000) in the Security Group, then browse to `http://<public-ip>:5000`.

Example: static website on S3

- Create an S3 bucket (unique name).
- Upload `index.html`.
- Enable **Static website hosting** in bucket properties.
- Allow public read through a bucket policy.
- Open the website endpoint URL.

Deployment options compared

Option	Effort	Control	Good for
EC2	High (you manage server)	Full	Custom apps
Elastic Beanstalk	Low (upload code)	Medium	Web apps, quick deploy
Lambda	Very low	Low	Event-driven small functions, APIs
ECS / EKS	Medium	High	Containers and microservices

11. Deploy AI and Analytics Workloads in the Cloud

Why use cloud for AI?

- On-demand powerful **GPUs** without buying them.
- Huge scalable storage for data.
- Ready-made managed ML platforms.
- Pay only while training or running.

ML life-cycle

Collect data -> Clean/Prepare -> Train model -> Evaluate -> Deploy -> Monitor

AWS services for AI / ML

Service	Use
SageMaker	Build, train and deploy your own ML models (notebooks, training jobs, endpoints)
EC2 GPU instances (P, G family)	Custom model training with full control
Amazon Bedrock	Use ready foundation models for generative AI
Rekognition	Image and video analysis (faces, objects)
Comprehend	Text analysis (sentiment, keywords)
Polly / Transcribe	Text to speech / Speech to text
Translate, Lex, Textract	Translation, chatbots, reading text from documents

Ways to deploy a trained model: real-time endpoint (SageMaker), batch inference, serverless (Lambda), or container on ECS/EKS. **Equivalents:** Azure ML / Azure OpenAI, Google Vertex AI.

Analytics (Big Data) workloads

Need	AWS service
Store raw data (data lake)	S3
Clean and transform data (ETL)	Glue
Run SQL on files in S3	Athena
Hadoop / Spark big data processing	EMR
Data warehouse	Redshift
Real-time streaming data	Kinesis
Dashboards and reports (BI)	QuickSight

Data sources -> Kinesis -> S3 (data lake) -> Glue (ETL)
| -> Athena / Redshift -> QuickSight dashboards
| -> SageMaker (ML models)

Best practices: use Spot instances for training to save cost, separate storage from compute, encrypt data, control access with IAM, monitor cost.

12. Introduction to DevOps

DevOps = Development + Operations. It is a culture and set of practices where developers and operations teams work together, using automation, to build, test and release software faster and more reliably.

EASY WAY TO REMEMBER: Before DevOps: developers 'throw the code over the wall' to operations, and problems start. With DevOps: one team, one pipeline, automatic delivery.

Why DevOps?

- Faster releases.
- Fewer bugs and failures.
- Quick recovery when something breaks.
- Better teamwork between Dev and Ops.

DevOps life-cycle (infinity loop)

Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor -> Plan

Key practices

Practice	Meaning
CI - Continuous Integration	Developers merge code often; every merge triggers automatic build and tests
Continuous Delivery	Code is always ready to release; a human approves production deployment
Continuous Deployment	Every passed change goes to production automatically (no manual step)
IaC - Infrastructure as Code	Create servers/networks using code files (Terraform, CloudFormation)
Configuration management	Automate server setup (Ansible, Puppet, Chef)
Monitoring and logging	Watch apps in production (Prometheus, Grafana, CloudWatch, ELK)
Containers	Package app with dependencies (Docker) and manage with Kubernetes
DevSecOps	Add security into every stage

DevOps tools

Stage	Tools
Source control	Git, GitHub, GitLab
CI/CD	Jenkins, GitHub Actions, GitLab CI, AWS CodePipeline
Build	Maven, Gradle, npm
Containers	Docker, Kubernetes
IaC / Config	Terraform, CloudFormation, Ansible
Monitoring	Prometheus, Grafana, Nagios, CloudWatch
Testing	Selenium, JUnit

Simple CI/CD pipeline

Push code to Git -> Jenkins/GitHub Actions build -> Automated tests -> Docker image -> Deploy to staging -> Approval -> Deploy to production on AWS -> Monitor

Agile vs DevOps: Agile focuses on iterative software development with customers. DevOps extends this to deployment and operations. **DevOps and cloud:** cloud gives on-demand environments and APIs, which makes automation (CI/CD, IaC) easy.

CALMS: Culture, Automation, Lean, Measurement, Sharing.

13. Last-Minute Cheat Sheet

Topic	One-line answer
Cloud computing	Using IT resources over internet, pay as you use
NIST 5 characteristics	Self-service, Broad access, Pooling, Rapid elasticity, Measured service
Deployment models	Public, Private, Hybrid, Community (+ Multi-cloud)
IaaS / PaaS / SaaS	Infrastructure (EC2) / Platform (Beanstalk) / Software (Gmail)
Virtualization	Many VMs on one physical machine using a hypervisor
Type 1 / Type 2 hypervisor	Bare-metal (ESXi, KVM) / Hosted (VirtualBox)
Scalability types	Vertical = scale up, Horizontal = scale out
Region / AZ / Edge	Geographic area / data centres in a region / CDN sites
EC2	Virtual servers (IaaS)
AMI	Template to launch EC2
Key pair	SSH login keys (.pem)
Security Group	Instance firewall, stateful
EC2 pricing	On-Demand, Reserved, Spot, Dedicated
Lambda	Serverless, event-driven, 15 min max, pay per request
S3 / EBS / EFS	Object / Block (one EC2) / File (many EC2)
RDS / DynamoDB	Managed SQL / Managed NoSQL
IAM	Users, groups, roles, policies; least privilege + MFA
CloudWatch	Monitoring and alarms
SageMaker	Build, train, deploy ML models
Git vs GitHub	Tool vs hosting website
git pull	fetch + merge
DevOps	Dev + Ops with automation; CI/CD, IaC, monitoring
CI vs CD	Integrate and test automatically / Deliver or deploy automatically

14. Practice MCQs

No.	Question	Answer
1	Which is NOT a NIST characteristic of cloud?	Fixed capacity (Cloud has rapid elasticity, so fixed capacity is not a characteristic)
2	Gmail is an example of?	SaaS
3	AWS Elastic Beanstalk is an example of?	PaaS
4	AWS EC2 is an example of?	IaaS
5	Which hypervisor runs directly on hardware?	Type 1 (e.g., VMware ESXi, KVM, Xen)
6	Which hypervisor is VirtualBox?	Type 2 (hosted)
7	Combination of public and private cloud is called?	Hybrid cloud
8	Which AWS storage is object storage?	S3
9	Which storage is block storage for EC2?	EBS
10	Maximum run time of one Lambda execution?	15 minutes
11	Which EC2 pricing is cheapest but can be interrupted?	Spot instances

No.	Question	Answer
12	Which file is used to SSH into a Linux EC2?	Key pair (.pem file)
13	Which port is needed for SSH? For HTTP?	22 for SSH, 80 for HTTP
14	What stores a template of OS + software for EC2?	AMI
15	Does a stopped EC2 instance cost anything?	No compute charge, but attached EBS is charged
16	Command to download a remote Git repo?	git clone
17	Command that does fetch + merge?	git pull
18	Safe way to undo a commit?	git revert
19	Which AWS service builds and trains ML models?	SageMaker
20	Which AWS service runs SQL queries on data in S3?	Athena
21	DevOps practice where every code merge is built and tested automatically?	Continuous Integration (CI)
22	Which tool is used for Infrastructure as Code?	Terraform / CloudFormation
23	Moving extra load from private to public cloud is called?	Cloud bursting
24	Which AWS service provides DNS?	Route 53
25	Which AWS service monitors metrics and sets alarms?	CloudWatch

Good luck with your exam! Revise tables, commands, EC2 launch steps and the SaaS/PaaS/IaaS comparison - these carry the most marks.